University Name

Department / School

Your PhD Study Report Title

Your Name

A study report submitted for the degree of
Doctor of Philosophy

Supervisor: Supervisor Name

June 29, 2026

Abstract

Write your abstract here. Summarize the research problem, the approach taken, the key findings, and the contribution of this study report in a single self-contained paragraph (typically 150–300 words).

Contents

List of Figures

List of Tables

Chapter 1

Understanding Reasoning

1.1 Reasoning Model

In LLM development, reasoning is defined through a practical engineering lens rather than a philosophical one. Reasoning refers to a model generating intermediate steps before delivering a final response. These steps may be:

- **Visible:** Presented to the user as a step-by-step explanation.
- **Hidden:** Enclosed in special tags (e.g., <think>...</think>) and processed internally.

This process is frequently called "Chain-of-Thought." While it resembles human internal monologue, the underlying mechanism is autoregressive token prediction based on statistical patterns. The goal is to encourage the model to allocate computational resources (tokens) to problem-solving.

Reasoning models excel at tasks where pattern recognition alone fails:

- Logical puzzles and multi-step math problems.
- Complex coding tasks.
- AI agent applications requiring task decomposition, planning, and error recovery.

1.2 Standard LLM Training Pipeline

To understand where reasoning models diverge, one must first understand the training pipeline in the Table 1.1.

Table 1.1: LLM Training Pipeline.

Stage	Process	Objective
Pre-Training	Trained on trillions of tokens of unlabeled text using massive GPU clusters.	Learn "raw language prediction" (next-token prediction) and develop emergent properties.
Post-training: Instruction Tuning	Supervised-finetuning (SFT) on labeled dataset.	Improve the model's ability to follow human instructions for specific tasks (summarization, translation).
Post-training: Preference Tuning	Reinforcement Learning with Human Feedback (RLHF).	Align model outputs with human stylistic choices and subjective preferences.

1.3 Methodologies for Enhancing Reasoning Capabilities

Improving reasoning involves specific techniques typically applied after or on top of the conventional pipeline.

1.3.1 Inference-Time Compute Scaling

This method improves performance at the moment of the user prompt without modifying the model's weights. It trades increased computation for higher accuracy using:

- Chain-of-Thought prompting
- Advanced sampling and voting procedures to identify the best candidate solution.

1.3.2 Reinforcement Learning (RL) for Reasoning

Unlike RLHF, which relies on subjective human judgment, RL for reasoning uses automated or environment-based reward signals.

- **Verification:** Rewards are given for verifiable correctness (e.g., a correct math answer or functional code).
- **Weight Updates:** This process modifies the model's weights through trial and error.

1.3.3 Knowledge Disitllation

This involves transferring reasoning patterns from a large, high-performing model ("teacher") to a smaller, more efficient "student" model. This is typically achieved via SFT using high-quality reasoning datasets generated by the stronger model.

1.4 Pattern Matching vs. Logical Reasoning

A critical distinction exists between how LLMs "reason" and how deterministic rule-based systems operate.

- Rule-Based Systems (Symbolic Logic): These follow strict, hand-crafted heuristics and formal rules. They guarantee logical consistency and correctness if the premises are true.
- Statistical LLMs (Pattern Matching): These identify statistical associations. For example, if a model says the capital of Germany is Berlin, it is recalling a high-probability association, not deducing it.
- Implicit vs. Explicit Reasoning: Conventional models like GPT-4o can simulate reasoning in familiar contexts because they have encountered similar scenarios in their training data.

However, they struggle with novel scenarios or highly intricate multi-step relationships because they do not explicitly track logic or contradictions.

1.5 Report Structure

The shift toward reasoning models is a major industry trend characterized by the following developments:

- **Competitive Landscape:** Major advancements include OpenAI's o1 (September 2024) and DeepSeek's R1 (January 2025). Other major players include Anthropic (Claude 4), Google (Gemini 2.5), and Alibaba (Qwen3).
- **Model Unification:** OpenAI's leadership has stated that GPT-4.5 (Orion) will be their last non-CoT model, with future goals to unify "thinking" models (o-series) and "standard" models (GPT-series).
- **Design Efficiency:** Reasoning is not always desirable. It is "overkill" for simple tasks like translation or basic summarization. Reasoning models are:

Chapter 2

Foundational Text Generation with Pre-trained Large Language Models

2.1 The Reasoning Model Development Framework

The transition from a standard LLM to a reasoning-capable system follows a four-stage mental model. Current focus remains on Stage 1: LLM Basics, which involves the initialization of a conventional LLM and the implementation of basic text generation.

Table 2.1: Basic Text Generation Pipeline.

Stage	Focus Area	Objective
1	LLM Basics	Load pre-trained weights and implement text generation functionality.
2	Measuring Performance	Evaluate response qualities using benchmarks and judgment-based metrics.
3	Reasoning Without Training	Explore inference techniques like self-refinement and advanced voting.
4	Reasoning With Training	Improve capabilities via Reinforcement Learning and Distillation.

Reasoning techniques are typically applied on top of a conventional (base) LLM. A "base" model is one that has undergone pre-training but has not yet been instruction- or preference-tuned. This approach allows developers to isolate which capabilities originate from the reasoning methods themselves versus the underlying model.

2.2 The Tokenization Pipeline

Tokenization is the critical bridge between human-readable text and the numerical data (Token IDs) required by neural networks.

The system utilizes a subword-based method known as Byte Pair Encoding. This allows the model to represent common words as single units and rare words as subword components (e.g., "Explain" may be split into "Ex" and "plain").

The Qwen3Tokenizer features a vocabulary of approximately 151,000 tokens.

- Large Vocabulary Pros: Allows more words to be represented as single tokens, resulting in shorter sequence lengths and lower overall processing time.
- Large Vocabulary Cons: Increases model size and the per-token compute cost for calculating next-token probabilities.

2.3 The Autoregressive Generation Process

Text generation in LLMs is autoregressive, meaning the model generates one token at a time, and each generated token is appended to the input for the next iteration.

2.3.1 Mechanics of a Single Iteration

1. Forward Pass: The model processes the full input sequence (e.g., 6 tokens).
2. Matrix Output: The model returns a matrix (e.g., 6 x 151,936). Each row represents scores for every word in the vocabulary.
3. Last Position Extraction: Only the prediction at the final position is used, as it indicates the next unseen token.
4. Greedy Decoding (Argmax): The system selects the single highest-scoring next token by finding the index of the largest value in the final row of the output matrix.

Chapter 3

Inference-Time Scaling

3.1 The Paradigm of Inference-Time Scaling

In the development of reasoning models, there are two primary strategies to enhance performance: increasing training compute or increasing inference compute.

3.1.1 Core Definition and Mechanics

Inference-time scaling involves allowing a model to do "more work" per question after it has already been trained. Rather than changing the model's weights, this approach utilizes the same trained model to generate higher-quality responses by:

- **Generating more tokens:** Allowing the model to "think" longer.
- **Sampling multiple answers:** Generating a variety of potential solutions to find the most robust one.
- **Successive refinement:** Iteratively reviewing and improving an initial response.

3.1.2 The Scaling Trade-off

There is a direct correlation between the resources spent during inference and the resulting accuracy. As illustrated in the source data, spending more compute on the fly results in a sharper upward curve for benchmark accuracy. The primary disadvantage is that every additional token requires another forward pass through the model, increasing latency, compute cost, and API expenses.

3.2 Primary Methodologies for Improved Reasoning

The following three methods represent the practical paradigms for scaling reasoning performance without retraining the model.

3.2.1 Method 1: Chain-of-Thought (CoT) Prompting

CoT prompting is a sequential technique that modifies the input prompt to encourage the LLM to generate a "reasoning chain" or a step-by-step explanation before arriving at a final answer.

- **Implementation:** Simple instructions such as "Explain step by step" or "Let's think step by step" are appended to the prompt.
- **Mechanism of Improvement:**
  - **Self-Correction:** Walking through steps allows the model more opportunities to correct its own trajectory.
  - **Pattern Alignment:** Most high-quality math and logic datasets contain detailed solutions; CoT aligns the model with these learned patterns.
- **Limitations:** CoT is not universally beneficial. On simple problems, it may lead to "overthinking," where the model generates erroneous explanations that mislead it toward a wrong conclusion.

3.2.2 Method 2: Parallel Sampling and Self-Consistency

This method involves generating multiple independent responses for the same query and using a "voting" mechanism to select the most frequent answer.

- **Implementations:** The model generates N diverse candidate answers. The system then selects the most common final result.
- **Synergy:** This technique is often combined with CoT prompting to generate multiple reasoning chains, significantly increasing the probability of reaching the correct conclusion.
- **Production Use:** This approach is a staple of advanced systems, including high-tier models like Claude 4.

3.2.3 Method 3: Iterative Self-Refinement

A more advanced approach where the model reviews its own reasoning and answers across multiple steps, progressively improving the output.

3.3 Technical Controls for Output Diversity

To enable methods like Self-Consistency, the model must be able to generate diverse responses rather than the same response every time. This requires moving beyond "greedy decoding" (always picking the highest-probability token).

3.3.1 Logits and Probability Distribution

The generation process involves three stages:

1. **Tokenization:** Converting text to token IDs.
2. **Logit Generation:** The model computes raw scores (logits) for every possible next token in the vocabulary (e.g., 151,936 unique tokens).
3. **Token Selection:** Selecting the index with the highest score.

3.3.2 Temperature Scaling

Temperature is a parameter that adjusts the "sharpness" of the logit distribution before it is converted into probabilities via the Softmax function.

Table 3.1: Temperature Settings and Their Effect on Output.

Temperature Setting	Effect on Distribution	Model Behavior
0.0 (Greedy)	Maximum sharpness	Always picks the single most likely token.
Low (< 10)	Sharper distribution	Increases confidence; the gap between the top token and others grows.
1.0	No change	Uses the model's original raw
High (> 1.0)	Flatter distribution	Increases diversity; loweer-ranked tokens become more likely to be selected.

3.3.3 Multinomial Sampling

Instead of always selecting the maximum logit, multinomial sampling draws a token at random based on the weighted probabilities. This randomness is essential for generating the diverse candidate answers required for Self-Consistency.

3.4 Balancing Diversity with Coherence: Top-p Filtering

High temperature settings can occasionally cause the model to sample "weird" or nonsensical tokens (e.g., sampling "mistress" when the query is "The capital of Germany is..."). To prevent this, Top-p Filtering (also known as Nucleus Sampling) is implemented.

3.4.1 The Four-Step Top-p Process

This filter ensures that the model only samples from the most plausible tokens:

1. **Sort:** Arrange token probabilities in descending order.
2. **Cumulative Sum:** Calculate the running total of probabilities.
3. **Apply Threshold:** Select the smallest subset of tokens whose cumulative probability exceeds the threshold $p(e.g., p = 0.9)$.
4. **Renormalize:** Rescale the remaining probabilities so they sum to 1.0, creating a new, valid distribution for sampling.

By dropping low-probability tokens, the model maintains coherence while still allowing for the variability needed for advanced inference scaling strategies.

Chapter 4

Preliminary Results

4.1 Results

Table 4.1: Summary of preliminary results.

Method	Metric A	Metric B
Baseline	0.00	0.00
Ours	0.00	0.00

4.2 Discussion

Interpret the findings and relate them to Chapter 2.

4.3 Limitations

State the current limitations.

Chapter 5

Conclusion and Future Work

5.1 Summary

Restate the problem and summarise your progress.

5.2 Future Directions

- Next step one.
- Next step two.

5.3 Timeline and Milestones

Table 5.1: Planned timeline.

Period	Milestone
Months 1–3	—
Months 4–6	—
Months 7–9	—

Appendix A

Supplementary Material

Place additional derivations, code, or data here. Appendix chapters are lettered automatically because of `\appendix` in `main.tex`.